

LAT_UDC

Physics-Grounded Generative UAV Design

Study Report

Architecture · grounding & the physics sanctuary · how the sanctuary is built · the V1-V3 optimizer and the four modes of V3 · backends & artifacts · the abstraction thesis · and an honest account of what it can and cannot do.



Date: 2026-08-17

Repository: github.com/Soumyadeep00git/LAT_UDC

Prepared with Claude (Opus) · every figure regenerated from the codebase

Contents

§1	Architecture — the hourglass	3
§2	Grounding & the physics library	4
§3	How the sanctuary is built	5
§4	Decoupling — the dimensional kernel	6
§5	The optimizer — V1 to V3 (flowchart)	7
§6	The four modes of V3	8
§7	The seeker case — meta-requirement & decoupling	9
§8	Backend pipeline & wiring	10
§9	The files produced	11
§10	Algorithm comparison	12
§11	On abstraction — the thesis	13
§12	Honest scope — can / can't	14

Executive summary

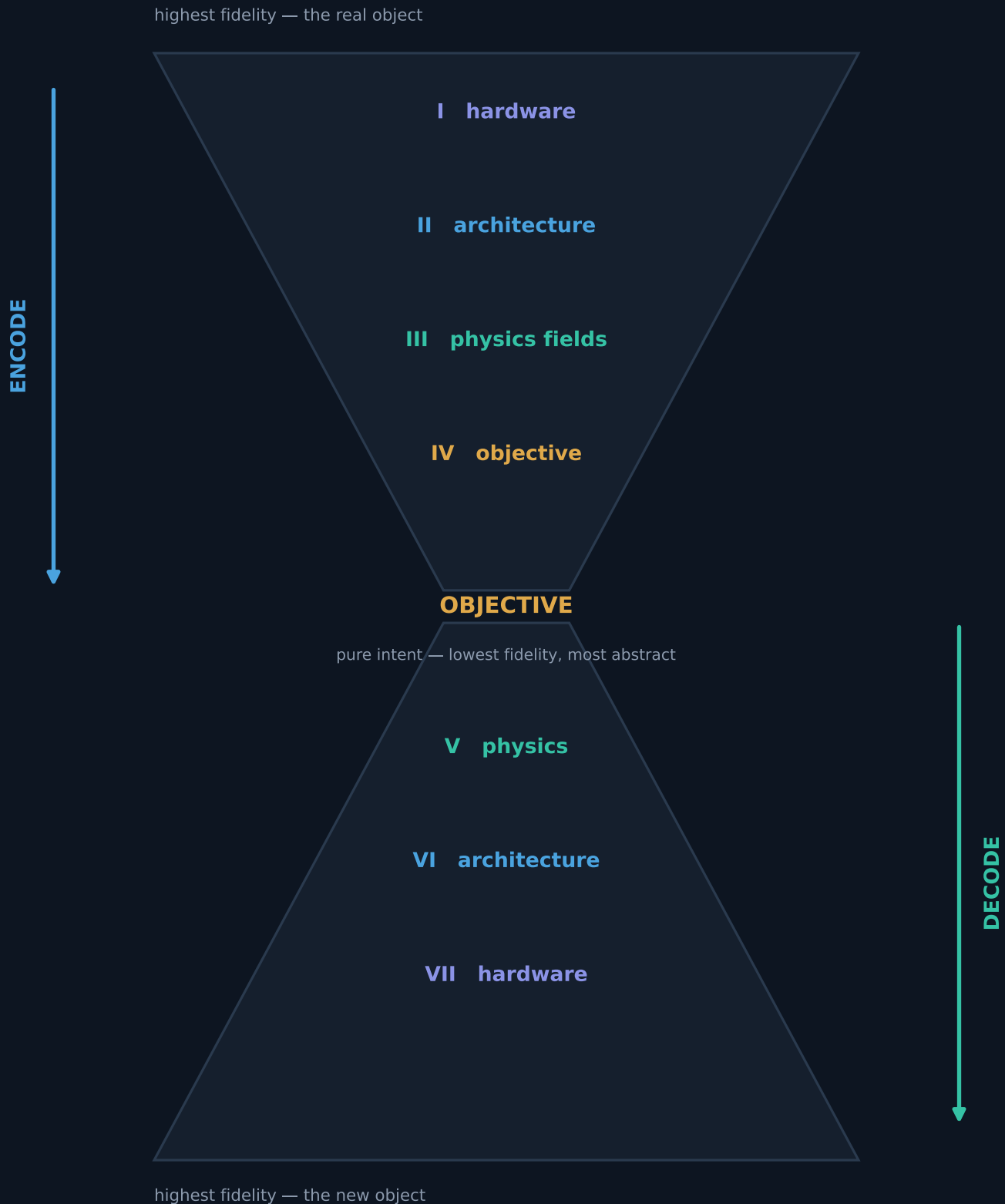
LAT_UDC turns a mission into a buildable vehicle and back: it encodes a concrete design down to intent, then decodes intent into a new design. Physics is not hard-coded per part — each design quantity is GROUNDED to a curated law, and law forms can even be re-derived from first principles. A three-tier optimizer tunes params (V1), rearranges structure (V2), and abstracts the function itself (V3). The engine is domain-general; the counter-UAS interceptor is instance one.

Headline result (this run, headless, validation PASS 8/8):

6-rotor · electric · propeller · ArduPilot/CubeOrange — a_max 6.6 g · v_max 100 m/s · 59 min

MAX INTERCEPT HITS = 7 / 12 (from baseline 1). The wall is the SEEKER, not thrust —
every airframe tier plateaus at 7/12; the binding limit is sensing.

ENCODE abstracts a concrete vehicle down to intent; DECODE regenerates one from intent



Two worlds run through the machine: the DARK (abstraction / law space — infinite, where it imagines) and the LIGHT (the coupled solve — definite, where reality answers yes or no). V3 is the traffic between them; fidelity is highest at the two ends and lowest at the objective, the narrow waist.

attaching real physics to a design quantity — and how the library is searched

What grounding is

Grounding binds a design quantity (a subsystem's function, e.g. "thrust") to a NODE in the physics library — a specific law, with the variables it needs, a validity envelope, and a descent to fundamentals. Once grounded, a design is no longer numbers: it is a graph of real physics with a traceable chain to conservation laws. Grounding is what makes the whole engine falsifiable.

Namespace discipline (no fuzzy matching):

a name is PHYSICS only if it (or its canonical form) is in the library vocabulary; otherwise it is a "math" parameter to be solved. That hard boundary — `library.classify()` — is why nothing is invented.

How the library is searched (`library.ground_quantity`)

1 canonicalize

`vocabulary.canonical(name)`: collapse synonyms (rotor_thrust -> thrust)

2 look up

`CANON_BY_QUANTITY[canonical]` -> candidate ids (fallback `A.BY_QUANTITY[name]`)

3 disambiguate

polysemy (rotor vs jet vs rocket thrust): pick the node whose required variables best OVERLAP the subsystem's own `physics_vars` -- exact overlap, no fuzzing

4 descend

`library.descent(node)`: trace to fundamental roots -- the validation chain

5 neighbours

`radicality.alternatives(quantity, node, radius)`: sibling embodiments within a crossing budget -- the candidates V2/V3 may cross to

Worked example — grounding "thrust" on the propulsion subsystem:

`canonical(rotor_thrust)=thrust` -> candidates {rotor, actuator-disk, jet, rocket, nozzle ...} -> `physics_vars` {air_density, disk_area, induced_velocity} overlap ==> `rotorcraft_bemt.rotor_thrust`.

the physics library, low level to high level — and how the algorithm links to it

Low level — one node

1,069 nodes. Each is an immutable ArcNode (physics_archive.py):

id	aerodynamics.aircraft_range
quantity	range (indexed by canonical())
law	Breguet: $R = (V/gSFC)(L/D) \ln(W_i/W_f)$
provenance	fundamental derived model
level	system high mid low fundamental
domain	aerodynamics, classical_mechanics ...
requires	{ variable : exponent } (units fingerprint)
points_to	[parent ids it rests on] (the edges)

High level — organization

- 5 LEVELS: system -> high -> mid -> low -> fundamental (descent depth)
- grouped by DOMAIN (aero, mechanics, electro-optics, thermo, materials ...)
- one CANONICAL vocabulary collapses synonyms (vocabulary.canonical)
- three indices built at import:
CANON_BY_QUANTITY, BY_QUANTITY,
VARIABLE_VOCAB (the known names)

How it is connected



Same edges, two readings: DOWN (points_to) validates; ACROSS (undirected) measures how far a swap crosses.

How the algorithm links to it

classify(name)	physics vs math — the hard boundary
ground_quantity(name, vars)	bind a subsystem's function to a node (canonical + overlap)
descent(node)	V3d / validation — trace the law to its roots
alternatives(qty, node, r)	V2 / V3 imagine — siblings within the radicality budget

Built by agents, curated, then frozen:

a swarm proposed nodes; smoke-tests gap-filled bindings (bindings_patch.py); the result is a fixed, generated archive — no runtime fuzzy matching. New physics is ADDED as nodes, never guessed at.

the library holds law forms; but forms are derivable from first principles — only constants aren't

Physics without the library (V3d)



From the units of the variables alone, the kernel derives the FORM of a law — the only dimensionally legal way the quantities can combine. A resolved field solve then supplies the one thing units cannot: the constant. Together they assemble a complete law with the physics library never touched.

Derived from units alone (verified):

rotor thrust

$$T = C \cdot \rho \cdot D^4 \cdot \Omega^2$$

aerodynamic drag

$$F = C \cdot \rho \cdot v^2 \cdot A$$

hover power (momentum)

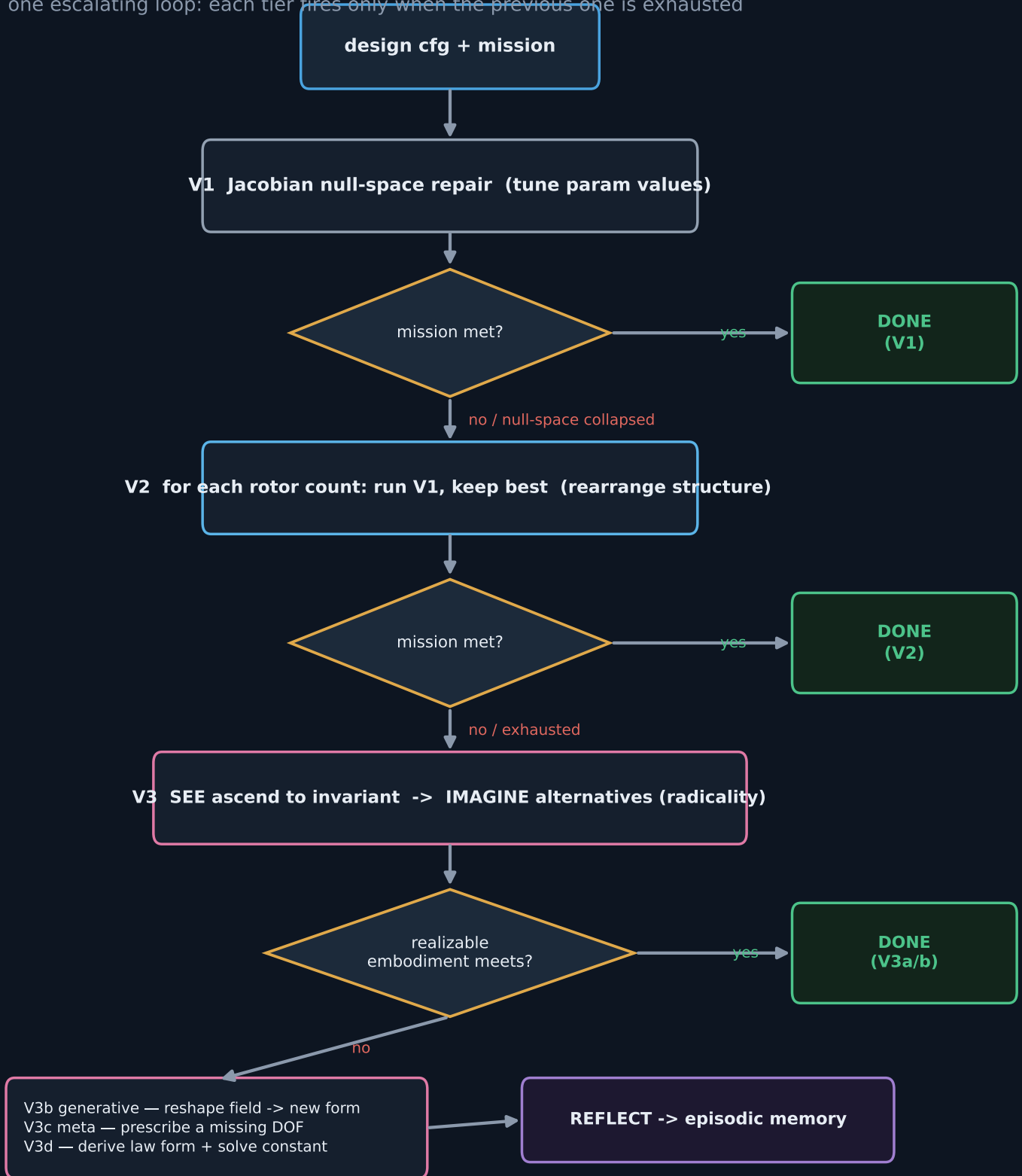
$$P = C \cdot W^{1.5} \cdot (\rho A)^{-0.5}$$

V3d proven end-to-end for drag: form (units) + $C_d = 0.48$ (CFD, 27k cells) $\rightarrow$ drag = $0.2414 \cdot \rho v^2 A$

The honest boundary

- Dimensional analysis gives the FORM/scaling — never the constant C or a function $f(Re, \dots)$.
- The constant comes from a field solve, which itself rests on measured MATERIAL constants
- (viscosity, moduli): the residue does not vanish — it descends to the few properties of matter.
- A full law is $C(Re)$: a sweep of solves; only an exact (DNS) solve is purely first-principles.
- So the library demotes to a CACHE of constants; the forms are generated on demand.

one escalating loop: each tier fires only when the previous one is exhausted



One metric — radicality — measures how far each tier moves (0 for V1; graph distance for V2; ascent altitude for V3). Escalation is triggered by exhaustion: when the null space collapses, climb a tier.

select · generate · prescribe · derive — with their motivation and status

V3a compositional

real

motivation: select an existing embodiment from the library

result: collapses to V2 (only realizes registered models)

V3b generative

prototype

motivation: dissolve to a field, reshape it, RE-EMBODY a new form

result: produced the ducted ring — a new form, UNVALIDATED

V3c meta-requirement

real (hint)

motivation: over-constrained field -> prescribe a MISSING DIMENSION

result: "seeker needs a scan DOF" — decoupled from the gimbal

V3d decoupling

real (drag)

motivation: derive the law FORM from units + constant from a solve

result: a complete library-free drag law

V3 is not one thing. As a design-point optimizer it folds into V2 unless it invents (V3b) — and its inventions are not yet validated. As an abstraction engine it genuinely exceeds V1/V2: it generates new forms, prescribes missing dimensions, and derives physics from first principles.

when a field is over-constrained, prescribe a missing DIMENSION — separate from the hardware

Two things 'decoupling' means

physics vs library (V3d)
form from units; constant from a solve;
library -> a cache of constants

physics-requirement vs hardware (V3c)
the field needs a new DIMENSION (motion);
the actuator that supplies it is a separate choice

The seeker wall — why interception plateaus at 7/12

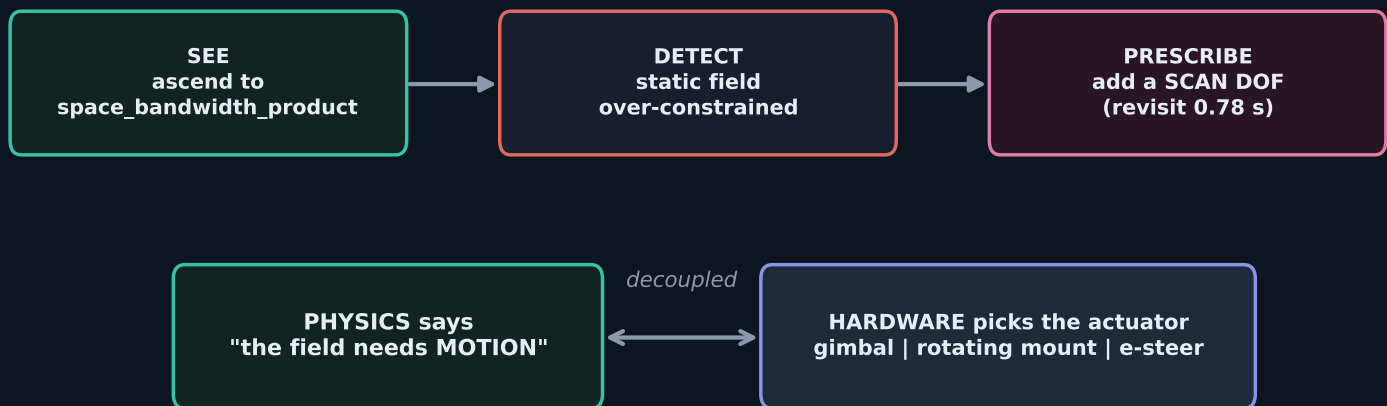
A fixed camera obeys a conserved budget (space-bandwidth / etendue): detection range x instantaneous coverage is fixed. It cannot have both long detection AND a wide field of view.

detect 2500 m -> needs 7.7 deg FOV

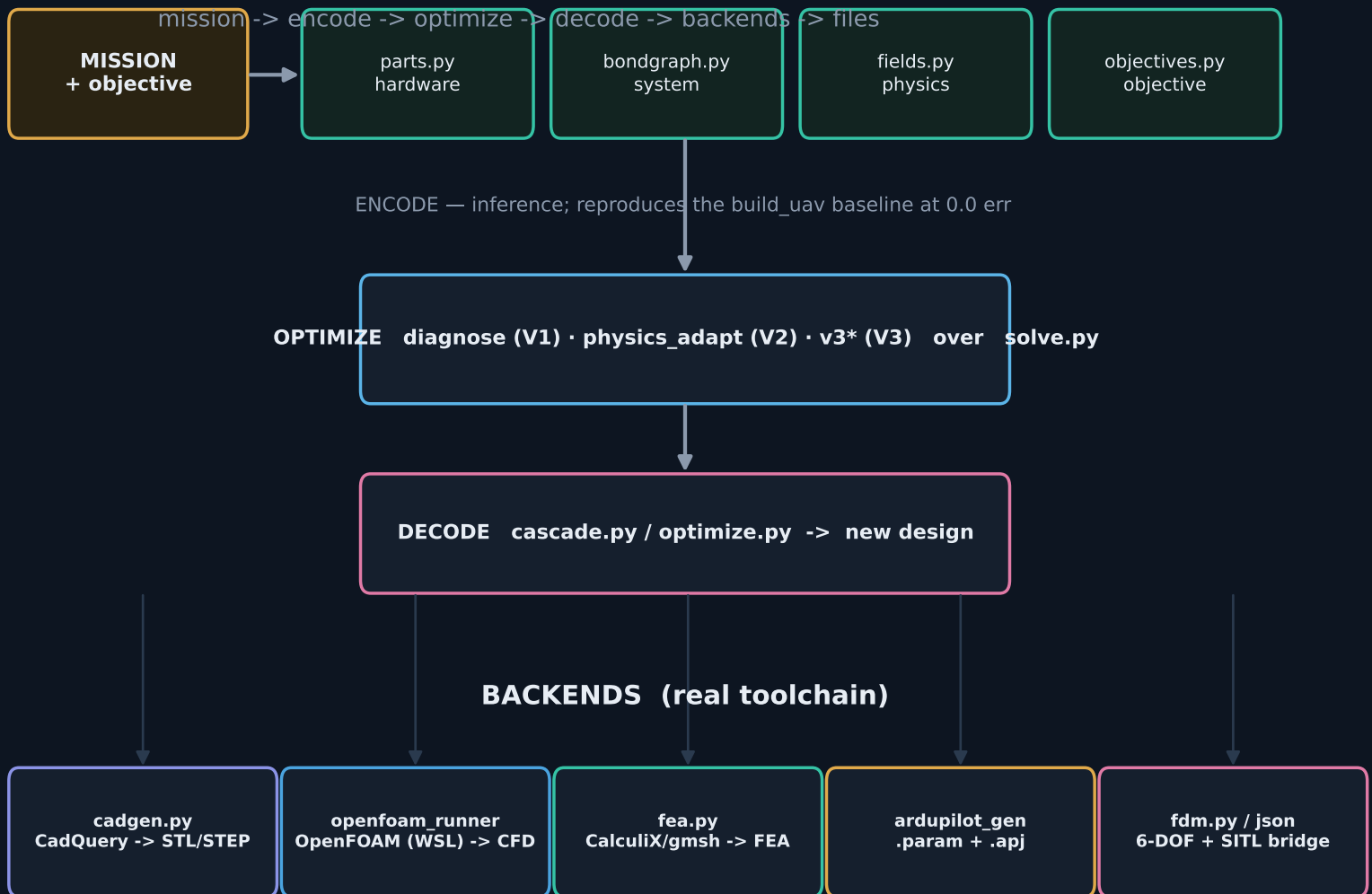
mission needs a 60 deg search cone

static camera: OVER-CONSTRAINED (infeasible)

What V3c did — grounded, then decoupled



V3 discovered a missing DIMENSION (motion), grounded in etendue, and prescribed it WITHOUT naming a gimbal — the difference from V1 (a missing value) and V3b (a missing form). Honest: this is the right question, decoupled — a hint, not a built scanning seeker; realizing it and re-scoring 7/12 is next.



Deficit-driven dispatch

A field inside its reduced-model validity uses the fast model; when it leaves validity (tip-Mach, disk-loading), it is handed to the external solver. The radicality leash is a proxy for validation strength — it lengthens exactly as far as the solve can be trusted.

one command (validate_pipeline.py) -> a complete buildable package

build_interceptor/

hardware/ specimen.step, specimen.stl

CAD B-rep + mesh

cfld/of_flow_case/

OpenFOAM: system · constant/polyMesh (27k cells) · 0/ + result

fea/ arm.inp, arm.msh, arm_result.json

CalculiX deck + gmsh mesh + solved beam FE

ardupilot/ specimen.param

41 generated params (frame, battery, ESC, seeker, failsafes)

ardupilot/ arducopter.apj (1.50 MB)

official CubeOrange firmware (downloaded, sha256)

ardupilot/ firmware_manifest.json

board id, git id, source URL, checksum

MANIFEST.json

design + capabilities + per-artifact provenance

Provenance is tagged per artifact — nothing fabricated:

- generated: STL/STEP (CadQuery), .param (from the design), the FEA deck + mesh
- solved: CFD drag (OpenFOAM RANS), beam-FE stress
- official release: the .apj (real ArduCopter firmware — not tool-authored; cannot compile here)

same mission; what each tier changes, its mechanism, result & status

tier	changes	mechanism	result (a>=5g,v>=26,e>=16)	status
baseline	-	-	UNMET (2.54 g)	-
V1	param values	Jacobian null-space repair	MET · 3.49 kg (4 rotors)	real
V2	+ rotor count	discrete search wrapping V1	MET · 3.04 kg (3 rotors)	real
V3a compositional	+ select embodiment	ascend->imagine->select	= V2 (collapses)	real
V3b generative	+ generate embodiment	reshape field -> re-embody	8.4g/73min/2.1kg	PROTOTYPE
V3c meta-req	+ a missing DOF	over-constraint -> etendue	seeker: 'add scan DOF'	real (hint)
V3d decoupling	+ derive the physics	dim. kernel + field solve	drag = $0.24 \cdot \rho \cdot v^2 \cdot A$	real (drag)

Reading it

- V1 does the real lifting; V2 beats it (lighter, feasible) by adding count on top of tuning.
- V3a folds into V2 — pure selection is not new. V3b/c/d are where V3 exceeds V2 (new form,
- new dimension, derived physics), but V3b is unvalidated and V3c is a hint, not a built part.
- On the interception score every airframe tier plateaus at 7/12 — the wall is the seeker, not thrust.

the idea the whole engine is reaching for, and why V3 is built the way it is

Two worlds. The PHYSICAL is what exists — finite, definite, the LIGHT where reality answers. The ABSTRACT is what could otherwise be — infinite, open, the DARK where a mind imagines. Abstraction is parasitic on the physical: you cannot abstract from nothing. Creation lives in the dark; correction lives in the light; and neither alone is enough — untethered imagination is only fantasy.

Abstraction = free the FUNCTION from its FORM ("free life from living").

To abstract a rotor is to ascend from "this rotor" to "produce reaction force by accelerating mass" — the invariant, embodiment-free. Once you hold the function, you are free to re-embody it as anything physics permits. The "One" the engine reaches for is not a final equation but a VERB: the operation of abstraction $\leftrightarrow$ embodiment itself. Building V3 is an attempt to instantiate that verb.

The ladder, in one line each

V1 change the value	move x within a fixed function f — tune params
V2 rewire the entities	change the number/links — the structure of f
V3 rewrite the function	reshape the field / add a dimension / derive the law — change f itself

Toward forming abstractions (not only traversing them)

The library is traversed; true abstraction FORMS new invariants. The loop is abstract $\rightarrow$ invent $\rightarrow$ reflect: retrospect over an episodic memory, extract an invariant nobody encoded, mint it. A drive stays fixed; the METHOD becomes revisable — the agent learns to mend its own way of seeing. Meta-requirements (a missing dimension, like the seeker's motion) are this faculty finding not a better value or form, but a needed axis the design did not have.

The honest ceiling

Imagination is free; realization is the wall. But validation propagates UP a sound derivation — validate the small physics and the higher inherits it — so the leash lengthens exactly as far as the derivation stays sound and in-envelope. Freedom is earned by grounding, not by removing the leash.

the line between validated, prototype, and frontier

CAN (real & verified)

- + mission -> buildable package, end to end (STL/CFD/FEA/.param/.apj); 8/8 validation
- + V1 params + V2 count: genuinely different, V2-dominant results
- + encode inference reproduces the hand-built baseline exactly (0.0 err)
- + reduced physics for 6 disciplines incl. a modeled EO/IR seeker
- + CFD (OpenFOAM/WSL) for airframe drag; FEA deck + solved beam; 6-DOF FDM + SITL bridge
- + V3: imagine alternatives, reshape a 2-D field, hint a missing DOF, derive law forms from units
- + V3d: a complete library-free drag law (form from units + constant from CFD)

CAN'T / DOESN'T (yet) — flagged, not hidden

- x validate V3's invented embodiments — the ducted ring is an UNVALIDATED deficit model
- x rotor-thrust CFD (only body drag); solve arm.inp with ccx (1-D beam); compile the .apj (no ARM toolchain)
- x fly live SITL (bridge verified in loopback only); fold CFD drag back into the caps
- x V3 RESCUE a mission — it folds into V2 unless it invents, and inventions aren't trusted
- x choose its own objective; invent unencoded physics/embodiments; C(Re) sweeps
- x other platforms/powertrains; disciplines thermal / RF / radar / guidance-miss / wind

The one honest throughline

Imagination is free; realization is the wall. V3 can abstract, reshape, and derive — but a form is only trustworthy when a validated model or solve backs it. Because validation propagates up a sound derivation, validating the small physics validates the higher; the leash lengthens exactly as far as the derivation stays sound and in-envelope. That boundary is the real remaining work.